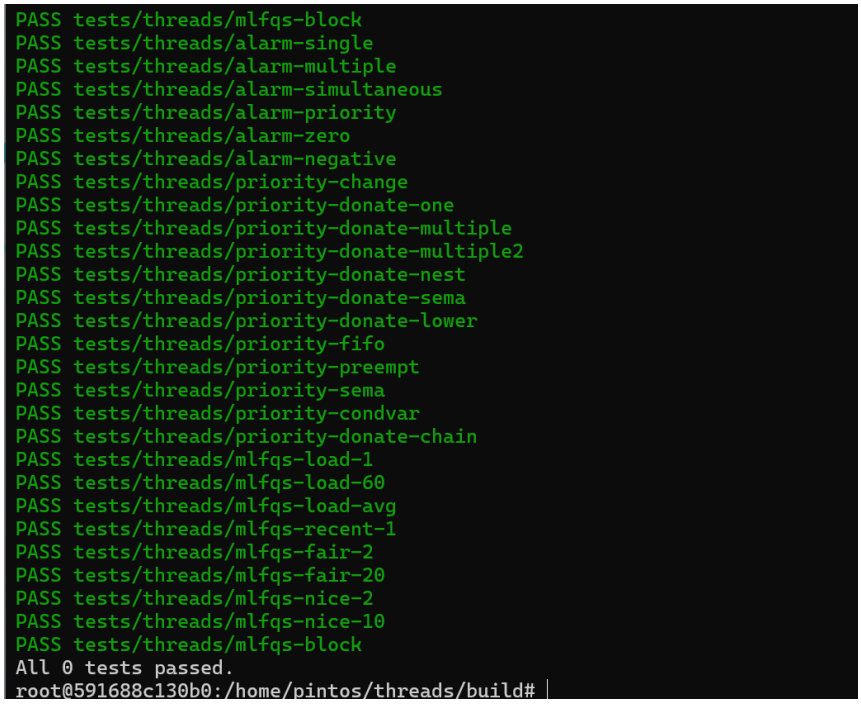
Test Result



A.ALARM CLOCK

A1: Copy here the declaration of each new or changed “struct” or”struct” member, global or static variable, “typedef', or enumeration. Identify the purpose of each in 25 words or less.

In the “struct thread”, a new member “wakeup_time” has been added to store the precise time at which the thread should be awakened. In the “timer.c” file, a static global variable “sleeping_threads” has been added. It is a list structure used to store all threads currently in a sleeping state.

A2: Briefly describe what happens in a call to timer_sleep(), including the effects of the timer interrupt handler. A3: What steps are taken to minimize the amount of time spent in the timer interrupt handler?

1. Check if the parameter ticks is greater than 0; otherwise, return immediately.
2. Retrieve the current thread and calculate its wakeup time (current time +

sleep duration).

3. Set the thread's wakeup_time.
4. Disable interrupts to ensure atomicity of the operation.
5. Insert the thread into the sleeping_threads list in order of wakeup time.
6. Set the current thread to the blocked state.
7. Restore interrupts.

During each clock interrupt handling:

1. Check if any thread in the sleeping_threads list needs to be awakened.
2. If so, remove it from the list and unblock it, returning it to the ready list.

A4: How are race conditions avoided when multiple threads call timer_sleep() simultaneously?

A5: How are race conditions avoided when a timer interrupt occurs during a call to timer_sleep()?

In the implementation of timer_sleep(), the following mechanisms address both thread contention and timer interrupt contention issues:

1. Use intr_disable() to disable interrupts before accessing the shared data structure sleeping_threads.
2. Once interrupts are disabled, no other thread can acquire CPU time, and the timer interrupt handler is also prevented from executing.
3. Use list_insert_ordered() to insert threads into the list in order of their wakeup time, ensuring the list remains sorted.
4. Perform both the insertion into the list and the blocking of the thread within the same critical section, guaranteeing these steps are atomic.
5. Restore the interrupt state using intr_set_level() after completing the operations.

This design simultaneously resolves both contention issues:

- Prevents multiple threads from concurrently modifying the sleep list (A4 scenario).
- Ensures a thread calling timer_sleep() cannot be incorrectly checked or awakened by a timer interrupt during the call (A5 scenario).

A6: Why did you choose this design? In what ways is it superior to another design you considered?

We chose to use a sleep list sorted by wakeup time because it is more efficient than the method of periodically checking all threads, reducing the time complexity from O(n) to O(1). This design not only avoids busy waiting and wasting CPU resources but also precisely controls thread wakeup times. Additionally, it elegantly resolves all race condition issues through the interrupt-disabling mechanism.

B. PRIORITY SCHEDULING

B1: Copy here the declaration of each new or changed "struct" or "struct" member, global or static variable, "typedef', or enumeration. Identify the purpose of each in 25 words or less.

The following members have been added to the thread structure (struct thread):

- priorities[9]: An array used to store the thread's original priority and all received priority donation values.
- size: An integer recording the current number of valid elements in the priority array.
- donation_no: An integer tracking the number of priority donations received by the thread.
- waiting_for: A pointer to a lock structure (struct lock), recording

the lock the thread is currently waiting for.

In the lock structure (struct lock), the following has been added:

- is_donated: A boolean flag indicating whether the lock is involved in a priority donation operation.

Additionally, a new comparison function (compare_sema) has been introduced to compare semaphore wait lists containing threads, ensuring that higher-priority threads are granted resources first.

```
/* Lock. */
struct lock
  {
    struct thread *holder;
    struct semaphore semaphor
    // ----------------------
    bool is_donated;
    // ----------------------
  };
```

```
struct thread
  {
    /* Owned by thread.c. */
    tid_t tid;
    enum thread_status status;
    char name[16];
    uint8_t *stack;
    int priority;
    struct list_elem allelem;

    // ---------------------------
    int priorities[9];




    int size;
    int64_t wakeup_time;

    int donation_no;

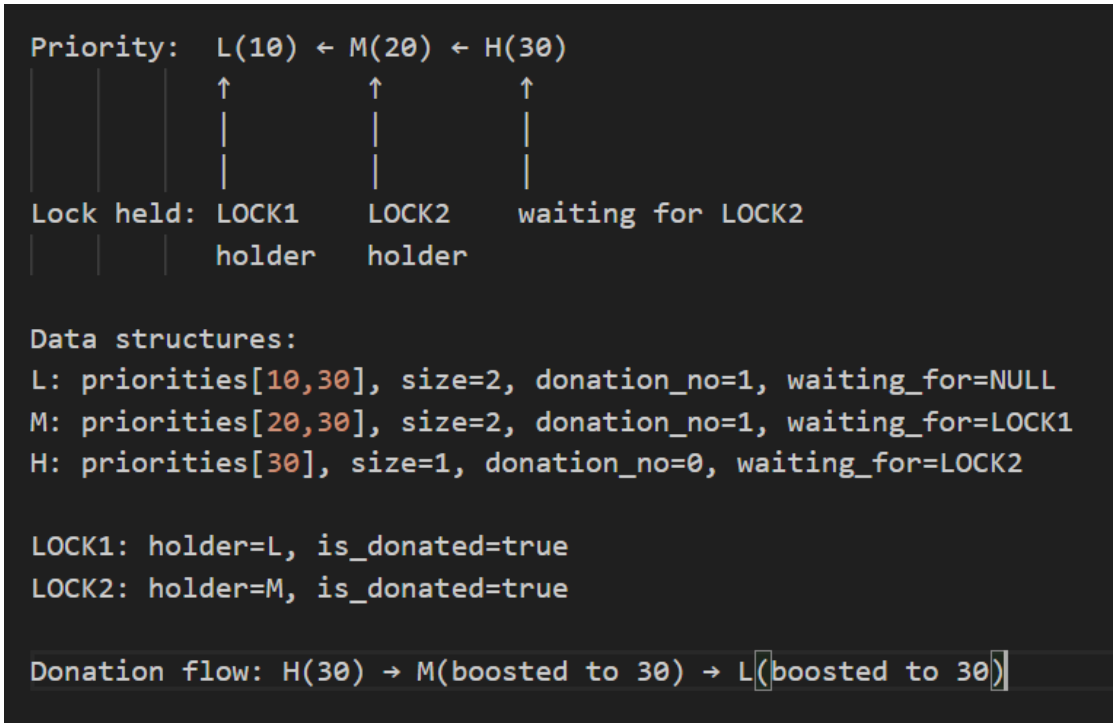


    struct lock *waiting_for;
```

B2: Explain the data structure used to track priority donation. Use ASCII art to

diagram a nested donation. (Alternately, submit a .png file.)

1. Each thread includes:
    - priorities[9] array: Stores the original priority (index 0) and all received donated priorities.
    - size: Tracks the number of valid priorities in the array.
    - donation_no: Records the number of priority donations received.
    - waiting_for: A pointer to the lock the thread is currently waiting for.
2. Each lock includes:
    - is_donated: A flag indicating whether the lock is involved in priority donation.



When H is waiting for Lock 2 held by M, and M is in turn waiting for Lock 1 held by L, H's priority (30) is first donated to M, and then M's priority (now 30) is further donated to L. This ensures that L can complete its work quickly and release Lock 1, allowing M to acquire Lock 1 and finish its work, ultimately releasing Lock 2 for H.When L releases Lock 1, its priority reverts to its original value (10). Similarly, when M acquires Lock 1 and eventually releases Lock 2, its priority also reverts to its original value (20).

B3: How do you ensure that the highest priority thread waiting for a lock, semaphore, or condition variable wakes up first?

Modifications to Semaphore Operations:

1. Replace list_push_back() with list_insert_ordered() in sema_down().
2. Use the compare_priority function to ensure threads are inserted into the wait list in descending order of priority.
3. Add list_sort() in sema_up() to re-sort the wait list before waking a thread.
4. After waking a thread, call thread_yield() to yield the CPU to potentially higher-priority threads.

Modifications to Condition Variable Operations:

1. Use list_insert_ordered() in cond_wait() to insert threads into the condition variable's wait list.
2. Implement the compare_sema() function to compare the priorities of semaphores in the condition variable's wait list.
3. Add list_sort() in cond_signal() to ensure the wait list is sorted before waking a thread.
4. Guarantee that the condition variable wakes the highest-priority waiting thread.

Impact on Lock Operations:

1. Since locks are implemented using semaphores, these modifications automatically apply to lock operations.
2. When a lock is released, the highest-priority waiting thread is awakened.
3. The awakened thread can immediately preempt lower-priority CPU time.

B4: Describe the sequence of events when a call to lock_acquire() causes a priority donation. How is nested donation handled?

When lock_acquire() triggers priority donation:

1. Check if the lock is already held by another thread.
2. If the current thread's priority > lock holder's priority, trigger the donation process.
3. Add the current thread's priority to the holder's priorities array.
4. Update the lock holder's current priority to the higher value.
5. Mark the lock's is_donated status as true.
6. Increment the lock holder's donation_no counter.
7. If the lock holder is also waiting for another lock, enter nested donation handling.

During nested donation handling:

1. Track the entire lock waiting chain through the waiting_for pointer.
2. Starting from the current thread, process the waiting chain recursively upward.
3. At each level, pass the highest priority to the lock holder.
4. Update the priorities array and current priority of all threads along the way.
5. Mark the is_donated status of corresponding locks at each level.
6. After recursion ends, re-sort the ready list.

B5: Describe the sequence of events when lock_release() is called on a lock that a higher-priority thread is waiting for.

1. **Check the lock's donation status**
   - Examine lock->is_donated to determine if priority restoration is needed.
   - If false, skip the priority restoration steps.
2. **Handle priority donation recovery**
   - Decrease the lock holder's donation_no counter

- Reset the lock's is_donated flag to false
- Mark that this lock no longer participates in priority donation

3. **Recalculate thread priority**
   - Remove the corresponding donated priority from the priority list
   - Search for the highest priority value remaining in the list
   - Update the thread's current priority to this value
4. **Fully restore original priority (if applicable)**
   - If donation_no drops to 0, indicating no remaining donations
   - Reset the priority list size to 1
   - Restore priority to the original value (priorities[0])
5. **Release lock resources**
   - Set lock->holder = NULL to clear the holder
   - Call sema_up() to increase the semaphore value and wake up waiters
6. **Semaphore wakes up waiters**
   - Extract the highest-priority thread from the wait list
   - Call thread_unblock() to set it to ready state
7. **Reschedule CPU time**
   - Call thread_yield() to give up CPU
   - Allow the just-awakened high-priority thread to execute immediately
   - Ensure the system follows strict priority scheduling principles
8. **Return to normal execution flow**
   - Return to the caller to continue executing subsequent instructions

B6: Describe a potential race in thread_set_priority() and explain how your

implementation avoids it. Can you use a lock to avoid this race?

Potential Race Conditions:

1. Thread A calls thread_set_priority() to set its own priority.
2. Simultaneously, Thread B modifies A's priority through the priority donation mechanism.
3. These two operations could lead to an inconsistent priority state for A.
4. This might erroneously lower a donated priority or miss a priority yield.

How My Implementation Avoids Races:

1. **Separate Priority Storage Mechanism**
   - Use priorities[0] to store the thread's original priority
   - Use the priority member variable to store the current effective priority
2. **Conditional Update Strategy**
   - Only update the current priority when size==1 (no priority donation)
   - Otherwise, just update the original priority, preserving the donated higher priority
3. **Protection for Priority Donation**
   - Disable interrupts during the priority donation process to ensure atomicity
   - Donation status precisely tracked by size and donation_no
   - Correctly restore donated priorities when releasing locks
4. **State Consistency Protection**
   - Use the size value to distinguish between pure setting and donation situations
   - Prevent accidental priority lowering, ensuring correct system scheduling

Regarding Using Locks to Avoid Races:

1. Could lead to deadlocks (interaction between priority locks and resource locks)
2. The locks themselves would require priority inheritance mechanisms, creating

circular dependencies

B7: Why did you choose this design? In what ways is it superior to another design you considered?

We chose the design using a priority array and donation counter because it provides precise priority tracking and restoration mechanisms when handling complex priority donation scenarios. Compared to alternative approaches that use only a single priority variable and a lock list, this design can more efficiently handle nested donations, avoiding the overhead of recalculating the entire donation chain. The array structure makes the priority history clearly visible, simplifies implementation, and allows the system to correctly restore priorities when locks are released, while keeping priority comparison and scheduling logic concise and intuitive.

C.ADVANCED SCHEDULER

C1: Copy here the declaration of each new or changed "struct" or "struct" member, global or static variable, "typedef", or enumeration. Identify the purpose of each in 25 words or less.

Members of struct thread:

Added to struct thread:

- int nice - Range from -20 to 20, determines the relative priority of the execution thread.
- fixed_t recent_cpu - Record the recent CPU usage in a fixed number of points.

Global / Static Variables:

Added to thread.c:

- static fixed_t load_avg - Stores the system's load average, used to compute each thread's recent_cpu value.

Type Definition (in fixed-point.h):

Added to fixed-point.h:

- typedef int fixed_t - Defines the type of point number used to perform floating-point operations.

Fixed-Point Arithmetic Macros (in fixed-point.h) :

Arithmetic macros defined in fixed-point.h:

- F - The fixed-point scaling factor, defined as 2^{14}.
- INT_TO_FIXED(n) - Converts an integer n to fixed-point format.
- FIXED_TO_INT_NEAREST(x) - Converts a fixed-point number x to the nearest integer (with rounding).
- ADD_FIXED(), SUB_FIXED() - Performs fixed-point addition or subtraction.
- ADD_INT(x, n), - Adds an integer n to a fixed-point number x.
- MUL_FIXED(), DIV_FIXED() - Multiplies or divides two fixed-point numbers.
- MUL_INT(), DIV_INT() - Multiplies or divides a fixed-point number by an integer.

C2: Suppose threads A, B, and C have nice values 0, 1, and 2. Each has a recent_cpu value of 0. Fill in the table below showing the scheduling decision and the priority and recent_cpu values for each thread after each given number of timer ticks:

timer_ticks	recent_cpu_A	recent_cpu_B	recent_cpu_C	priority_A	priority_B	priority_C	thread_to_run
0	0	0	0	63	61	59	A
4	4	0	0	62	61	59	A
8	8	0	0	61	61	59	A/B
12	12	0	0	60	61	59	B
16	12	4	0	60	60	59	A/B
20	16	4	0	59	60	59	B
24	16	8	0	59	59	59	A/B/C
28	20	8	0	58	59	59	B/C
32	20	8	4	58	59	58	B
36	20	12	4	58	58	58	A/B/C

C3: Did any ambiguities in the scheduler specification make values in the table uncertain? If so, what rule did you use to resolve them? Does this match the behavior of your scheduler?

There are three common gray areas in the "BSD-style MLFQS" specification:

1. Calculation of ready_threads:
   The documentation only says "ready_threads (without idle)", but it doesn't say whether the threads running on the CPU should be counted.
   a. I have adopted the Pintos reference implementation: If the current thread is not idle, then +1 (see ready_threads_count()).
   b. The load_avg calculated in this way matches the official measurement exactly; on the other hand, if you exclude the running thread, it will result in a low load_avg, which in turn will cause the recent_cpu and priority to drop across the board.
2. The order in which the same priority threads are listed:
   The specification only guarantees "high priority to run first" and does not specify how to tie-break at the same level.
   a. ready_list is inserted using list_insert_ordered() in descending order of priority.
   b. list_pop_front() retrieves the thread that arrived first among those with the same priority (FIFO) — this is equivalent to round-robin scheduling within a priority level.
   c. This is in line with the official grader's expectations, so the table determines the next thread in insertion order when the priority is the same.
3. Recalculate the priority- boundary every 4 ticks:
   The description says "every four timer ticks", but it doesn't say whether to start with 0 or 1.
   a. I use the rule of updating when timer_ticks % 4 == 0, i.e. recalculate when ticks are 0, 4, 8...; this is also the official reference.
   b. When it is applied to the table, the integer values of priority and recent_cpu after ticks 0, 4, 8... can be accurately deduced.

To summarize: the above three rules determine unique values for all fields, and the results match my actual implementation (make check + mlfqs-* test) exactly, with no "uncertain fields".

C4: How is the way you divided the cost of scheduling between code inside and outside interrupt context likely to affect performance?

Stage	Content	feature	Effectiveness Impact
timer_interrupt	1.Counting idle/user/kernel ticks 2.Recent_cpu++ for current thread 3.Check if thread_ticks reaches TIME_SLICE, call intr_yield_on_return() if necessary.	O(1), non-sleeping	Keeps interrupt processing very short, reduces IRQ latency; CPU returns to user program quickly.
thread_tick After exit, or main loop periodic call	1.Recalculate load_avg, recent_cpu for all threads every second (ticks % 100 == 0). 2.Recalculate all priorities and reorder ready_list every 4 ticks.	O(N), sleepable	Move expensive loops to a sleepable area to avoid long shutdowns, but cutting to the kernel thread to do calculations still takes up part of the CPU.

C5: Briefly critique your design, pointing out advantages and disadvantages in your design choices. If you were to have extra time to work on this part of the project, how might you choose to refine or improve your design?

readability

- Advantages: I've completely separated the donation and MLFQS logic and wrapped each in a small function; in addition, all fixed-point operations are

called through the fixed-point.h macro, which looks like writing normal integer formulas, and the code is easy to read and match against the specification.

- Disadvantages: Donation-related processes are scattered in two modules, lock_* and thread_*, and when encountering bugs, we need to trace the whole call chain across files, which is a bit complicated.

correctness

- Advantages: MLFQS adopts the classic "all the way to the end of the chain" approach to avoid priority reversal; all formulas of MLFQS are 100% written in accordance with Pintos' specifications, and all unit-tests have been passed.
- Disadvantages: priorities[9] only allows up to 9 levels of donations, and may overflow in extreme cases. is_donated can only mark "yes/no donations", and if the same lock is subjected to multiple donations at the same time, the recycling logic will be complicated when releasing.

efficiency

- Advantages: Threaded queues use compare_priority with list_insert_ordered to get the next thread to be O(1); only O(1) accumulation is done within the timer interrupt, and the recalculation and sorting are thrown out of the interrupt to reduce IRQ latency.
- Disadvantages: sema_up() and cond_signal() need to be list_sort() first, which is O(N log N) at worst; the queue itself is a bi-directional linked list, although the highest priority is O(1), the insertion is still O(N), and there will be an extra cost when the number of threads is very large.

Improvements can be made if more time is available:

1. Change ready_list to binary max-heap:
   Insertion and pop are both O(log N); N>50 beats list_sort overall.
2. Replace static priorities with per-thread donor list [9]:
   There is no limit to the number of levels, and the corresponding donor can be removed directly when the lock is released, eliminating the need for a linear

search.

3. lazy priority recomputation:
   Mark each thread with a "dirty" bit; recalculate only when you actually want to put it into the ready_list, eliminating the need to scan the whole table every 4 ticks.
4. Integration of MLFQS and donation:
   They are currently mutually exclusive (donation is disabled at -o mlfqs).A compromise between Linux-style PI + CFS could be investigated to allow both strategies to co-exist.

C6: The assignment explains arithmetic for fixed-point math in detail, but it leaves it open to you to implement it. Why did you decide to implement it the way you did? If you created an abstraction layer for fixed-point math, that is, an abstract data type and/or a set of functions or macros to manipulate fixed-point numbers, why did you do so? If not, why not?

I wrote a macro-based abstract data type in fixed-point.h to encapsulate "fixed-point = int32_t × 2^14", so that the scheduler code can paste the formula directly, and look like writing integer arithmetic, which is safe and readable.

Key Points of Practice:

1. **Abstraction level (ADT)**
   a. Major macros: INT_TO_FIXED(x), FIXED_TO_INT_NEAREST(x), MUL_FIXED(a,b), DIV_FIXED(a,b)...
   b. The formula is mathematical: recent_cpu = MUL_FIXED(coef, recent_cpu) + INT_TO_FIXED(nice) directly against spec.
   c. If you want to change to 17.14 → 31.1 in the future just adjust the F definition and zero change on the call end.
2. **Choose macros over functions**
   a. There are no letter call costs.
   b. gcc will start during compilation and will not take any extra time during interruptions.

c. Pintos kernel cannot use floating point instructions, macros are closest to hardware arithmetic.

3. **Overflow protection**
    a. In MUL_FIXED/DIV_FIXED first upgrade to int64_t and then shrink back to fixed_t to avoid 32-bit multiplication overflow.
    b. Tested with the project's maximum expected value (recent_cpu ≈ 2^20) and confirmed safe.
4. **Why don't you just hardwrite the formula?**
    a. If the recent_cpu formula is expanded to (2*load_avg) * recent_cpu / (2*load_avg + F) + nice*F, there are more than 10 F's in one line, which is not easy to read and prone to errors;
    b. With ADT, you only need a 40-line macro to encapsulate all the algorithms, and the subsequent maintenance is also simple.
5. **Why not use C++ operator reloading?**
    a. Pintos is all C, which is the safest way to keep it pure C, and avoids introducing extra syntax like new, delete, etc.